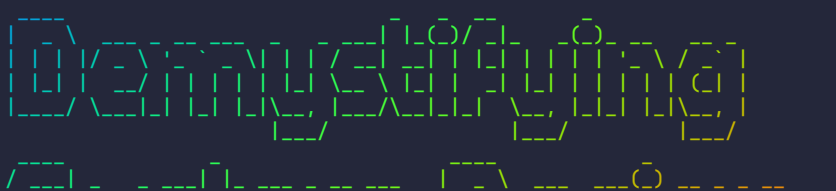
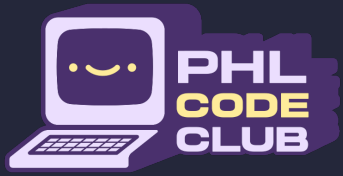


$: whoami

- Senior Software Engineer
- Co-founder and Chief Terminal Nerd @ PHL Code Club



- I live my life by 3 *C's*:
  - *Cats*
  - *Coding*
  - *Compassion*

Here is me with my cat, Fritzy =3

Setting Expectations

What this talk *is*:

- A primer on some common concepts/technologies in system design
  - Data stores, message brokers, scaling strategies, etc
- How to talk through your process and ask clarifying questions
  - Including backtracking, sometimes you need to rethink a decision
- An explainer on thinking in whole systems
  - Things like failure modes and temporality

What this talk *isn't*:

- An in-depth guide into any one technology or methodology
  - This is not an explainer on the internals of Postgres indexes
- A "how-to" style presentation
  - This is foundational knowledge, it's up to you to be curious!
- Entirely objective rhetoric
  - Some of what I'm gonna say is my *opinion*. I'll try to be clear about that

Outline

- Technologies
  - API Design
  - Common Scaling Patterns
  - Caching
  - Data Storage
  - Message Brokers
- Philosophy
  - Talk the Walk
    - Be sure to explain yourself as you are designing
  - Dazed and Confused
    - How to manage not knowing everything
  - Quick maths
    - Common calculations to keep in mind
  - Happy and unhappy paths
    - Errors happen, make sure to consider them
  - Time marches on
    - Understanding the role of time on systems

API Design: What Even Is an API

Application Programming Interfaces (APIs) are the public interface for programmatically interacting with other software. This can be an HTTP API, a CLI, a library, or even a wire protocol.

We will be focusing on *network APIs* in our examples. Common API types are:

- RESTful
  - This is the most common design you will see for HTTP APIs. Based on some of the foundational ideas of the early *World Wide Web*.
- GraphQL
  - Designed around graphs, developed at Facebook for handling social graphs. Allows for more flexibility for clients and built in validation for requests.
- RPC
  - "Remote Procedure Call" is effectively calling functions on remote machines. This makes the transport less important, so you can more easily move away from traditional HTTP.
- "Realtime"
  - Realtime APIs are designed around low latency bidirectional communication.

API Design: Common Design Goals

Your API is how users (or the clients they are using) interact with your services. Keeping clear boundaries between services, domains, or logical entities is key to making the API surface area simple and easy to understand. Having a well segmented API allows for easier changes in the future and less direct coupling of the contracts to the implementations. There are many intricacies to API design but some common things to think about are:

- Simplicity
  - Keep your design easy to reason about. Use consistent naming and access patterns. The *KISS* method is still used for a reason.
- Pagination
  - Any time you are returning large sets of data, split it into smaller sets with the ability to fetch individual sets. This reduces load on both your API servers and anything downstream that you are fetching data from.
- Access control
  - Utilize *authentication (authn)* and *authorization (authz)* to validate who clients claim to be and what they can access. Improper access control can lead to security incidents and/or data leaks.

API Design: RESTful

"RESTful" APIs follow particular conventions around HTTP methods and paths for handling routing. These are often JSON based HTTP APIs, however the format could be anything.

I put "RESTful" in quotes as this is not entirely accurate. Representational State Transfer is an architecture designed around sending representations of data as *hypermedia* and using links within that hypermedia to facilitate changes to the state. I usually just call these HTTP APIs as they are usually the default.

These HTTP APIs are the best supported option and are the easiest to cache. If you are making requests from a client of some sort (e.g. a browser or mobile app), these will be a great place to start.

API Design: GraphQL

GraphQL is an API architecture that has built-in type checking and introspection. As the name implies, GraphQL is designed around the graph of types and their dependencies. GraphQL uses a dedicated language for defining your schema and your operations.

GraphQL is a very flexible query language, as clients can ask for exactly the data they want. This lets you avoid *over-fetching* (getting more data than you need, wasting bandwidth and latency) or *under-fetching* (not getting enough data, forcing multiple requests).

Another great feature of GraphQL is there is a lot of tooling that lets you use your schema and operations to generate code for your applications. One huge time saver is having it generate types, as they will always be in sync with your API.

API Design: RPC

RPC APIs let you "call functions" on remote systems. Similar to "RESTful" APIs, the format of the data doesn't really matter. Some common RPC frameworks are *JSON RPC* using… JSON, and *gRPC* using Protobuf.

gRPC and protobuf are very common for *service to service* communication. As systems grow, you want fast and efficient communication between the different parts. The binary encoding of protobuf minimizes bandwidth and latency when sending information between these services.

RPC APIs let you model your API as a collection of function calls, rather than having to think in terms of HTTP requests with paths and verbs.

API Design: Realtime

Realtime APIs are best used when the standard request/response paradigm doesn't fit the actions being taken. For instance, if you are building a video chat service you wouldn't handle your streaming via a RESTful API.

Some common technologies you will see when dealing with realtime systems are:

- *Websockets*
  - Websockets are a standard for handling bidirectional communication inside web browsers. They have become more ubiquitous as you will likely not want to maintain a separate realtime API for your web clients vs native clients on desktop or mobile.
- *WebRTC*
  - WebRTC allows for peer to peer realtime communication on the web platform (e.g. web browsers). It is generally used for video and audio data, but also supports generic data as well.
- *Custom Wireprotocol*
  - For when the other two options don't fit your usecase or you need more performance you can design a custom wire protocol.

Scaling: Common Patterns

There are *two* main ways to scale web services:

- Vertical Scaling
  - Adding additional resources to a single server (CPU, RAM, or Storage).
  - **Example:** You have a *Virtual Private Server (VPS)* on a cloud provider running PostgreSQL. It is pretty consistently pinned at 90% memory usage. To vertically scale this you would upgrade the VPS instance to one that has more memory.
- Horizontal Scaling
  - Adding additional copies of a service.
  - **Example:** You have a web server running on a small virtual machine. During times of high traffic your latency starts to spike. This service doesn't store any data locally, everything is persisted elsewhere. Adding a second virtual machine with another copy of the service and putting them behind a *load balancer* will allow you to handle more requests.

Scaling: Load Balancing

As you get into scaling *horizontally*, you need to be able to route requests to one of *many* copies of your services. Doing so in a manner that doesn't overload any one server is called *load balancing*. The criteria you use for load balancing is entirely dependent on what the services do. However, some common strategies are:

- Round Robin
  - Just cycles through the servers to send some number of requests to one server before moving on to the next.
- Metric Based
  - Uses metric data from the services to decide when a service is at capacity then excludes it from accepting any more requests.
- Request Based
  - Routes to backends based on information in the request, such as headers, cookies, or ip address.
- Consistent Hashing
  - Similar to request based, but uses a hash ring to make adding and removing services less disruptive.

Scaling: Replication and Sharding

If you want to scale data *horizontally* you will need to be able to *replicate* and *shard* the data.

Replication refers to copying data between multiple locations (often separate servers or disks). This allows you to scale writes and reads independently.

Sharding is deterministically splitting data across multiple servers or disks based on a "shard key". This is more efficient than replication from a storage perspective, but requires more work from the database runtime.

Caching: What Is a Cache?

A *cache* is a separate store of data that is faster to read and write to than your primary memory. Your CPU has multiple levels of cache that vary in speed depending on how close they are to the registers.

We will primarily be talking about *in-memory* caches that use RAM as an intermediary to storage systems like databases. We will also touch on client side caches, which often store data on device to reduce round trips over the network.

Caching: When to Cache

Your first instinct might be to cache *everything*. While this is probably wicked fast, it will not only balloon your costs, but also introduces additional complexity.

The first step is to sit down and reason through the various paths to data inside your application. Then you can measure the number of requests, their latency, and errors like timeouts. You can use this data to better understand where you can improve data fetching.

Caching: How to Cache

There are some important aspects of managing caches to use them effectively. The first is your *eviction policy*. This is the rules for how your cache decides to delete data when the cache is full to make room for new entries. Some common eviction policies are:

- *Least Recently Used (LRU)*
  - This keeps track of when entries are accessed and evicts the entry with the longest time since it was last accessed.
- *Least Frequently Used (LFU)*
  - This keeps track of how often entries are accessed, evicting the entry with the lowest access count.
- *First In First Out (FIFO)*
  - This is effectively a queue. The first item to enter will be the first one to be evicted.

Another important factor is *time to live (TTL)*. This gives cache entries an "expiration date" of sorts. If data is older than its TTL, it will be considered *stale* should be treated as a cache miss.

Caching: Where to Cache

Another important aspect of caching is *where* your cache lives. Does it live on the client? Is it local to one specific server? Understanding where to cache is just as important as what you are caching.

- Client Caches
  - Web browsers have built-in caching support using HTTP headers. These caches will store static data locally and only re-fetch from the remote resource when the *TTL* expires or is manually evicted. You can also cache manually by storing information that can be reused in memory and writing code to manage stale data.
- App Caches
  - These are caches that are used by backend systems. This can be as simple as storing expensive to process data in memory, but you can also use standalone services such as *Redis* or *Memcached*.
- Distributed Caches
  - One way to complement the other caching strategies is to use distributed caches such as *Content Delivery Networks (CDNs)* or edge caches like *Fastly* and *Cloudflare*.

Schema Design

An important aspect of any system is having a clear shape for your data. This shape is called the "schema". Often this is used to describe relational databases, but it applies to any data that you deal with in your system. As such, you often end up with many definitions to describe the various services and data that you deal with.

When designing schemas in your systems it's good to keep in mind:

- flexibility
  - Keeping your schema flexible will save you a great deal of time in the early stages of ideation and building. However, you always want to balance it with a strong core around your data.
- Good Indexing
  - A huge performance hit comes from improperly indexed data. If you have to scan entire tables or collections when querying, that is a sign that you should reevaluate your indexing strategy.
- The Right Tool
  - Be sure to use a database that fits the shape of your data. At small scales this doesn't matter as much, but revisit when needed.

ACID (Not Basic)

ACID is a set of rules that apply to *transactions* in a database. A transaction is a set of changes that are applied to the state of your database.

- Atomicity
  - There are no partial states for transactions. They either get committed or aborted.
- Consistency
  - The state of a database is always valid and in accordance with the defined schema.
- Isolation
  - Transactions are completely independent and are not visible to other transactions.
- Durability
  - Once a transaction is committed that state can be recovered by the database.

Common Database Paradigms

Databases are not simply *SQL* vs *NoSQL*… They all have strengths and weaknesses. You need to be thinking about both the shape of your data, but also access and storage patterns to decide the right solution.

Some common database paradigms you will run into:

- Relational
- Document
- Wide-column
- Fulltext Search Engines

Databases: Relational

Relational databases are defined by structured sets of rows and columns called tables. These tables have strict data schemas that tell you what data types they can hold. They allow for creating relationships between columns of data which allow you to *join* tables together efficiently. They also are *ACID* compliant.

Relational databases are often hard to scale. We will get to *replication* and *sharding* in a bit, but generally it is easiest to scale them *vertically* to start.

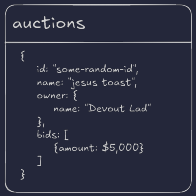
Common relational databases are PostgreSQL, MySQL, and SQLite. You may also come across "Enterprise" databases such as OracleDB and Microsoft SQL Server.

Databases: Document

Document databases are hierarchical sets of key-value pairs, similar to a key-value store, but they allow for nesting documents and support more complex data types.

One big feature of most *Document Databases* is that they are *schemaless*, meaning the data can be any shape when you store it. This allows for a lot of flexibility. However, this comes at the cost of consistency guarantees, especially since they are often not *ACID* compliant. These are often easy to scale *horizontally* due to the looser requirements around consistency.

Examples of *document databases* include MongoDB and CouchDB.

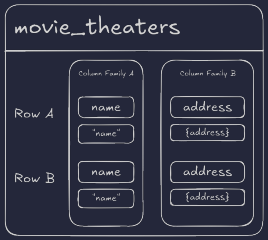


Databases: Wide-Column

Wide-column databases use flexible columns that can be spread across multiple servers or database nodes, using multi-dimensional mapping to reference data by column, row, and timestamp.

Unlike relational databases the rows in wide-column databases can all have different numbers of and names for columns. These are often very easy to scale *horizontally* and can handle heavy write loads on account of how the data is mapped by the storage engine.

Examples of *wide-column databases* include Cassandra and ScyllaDB.

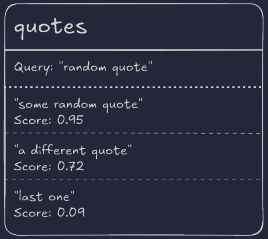


Databases: Fulltext Search Engines

Fulltext search engines ingest data from other sources and index it in such a way that makes searching text (hence the name) and ranking results fast and efficient.

These are often *secondary* to a primary database, such as a relational or document database. You will often have some sort of process or service that syncs your *main* data store with your *fulltext search engine*.

Some examples of *fulltext search engines* are ElasticSearch, OpenSearch, and Typesense.



Object Stores

Databases are really good at handling structured and semistructured data, usually text. If you want to deal with large sets of binary data, you will want to reach for an *object store* (sometimes called a *blob store*).

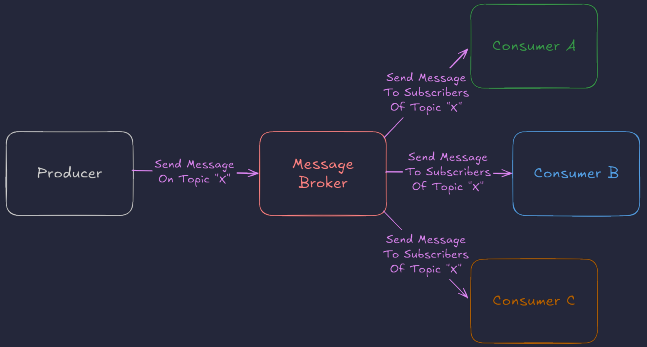
These often act similar to distributed filesystems. Your data is given a location and a small amount of metadata (the size of the file, when it was created, etc). You are then given some sort of API to read and write binary data to these locations.

Most cloud providers will have object stores, but the most ubiquitous is Amazon S3. It is so common that often times other services will have S3 compatible APIs.

Message Brokers: Overview

Message brokers are systems that take in messages from *producers* and send those messages to *consumers*. Often these take the form of distributed "message queues". They allow you to send off data without caring about the result (either immediately or ever).

Some common message brokers are *Rabbit MQ* and *Apache Kafka*.



Message Brokers: Patterns

Message brokers allow for more flexible data flows over the normal "request response" pattern. By utilizing message queues you can asynchronously handle work, or distribute it to multiple workers. There are a couple common patterns you might run into:

- *Asynchronous processing*
  - This pattern allows you to fire off a message to kick off some processing without *presently* needing the result.
- *Publish Subscribe (Pubsub)*
  - Very useful for propagating data out to many services at once. Often you will have a *topic* on your message brokers that consumers can subscribe to.
- *Fan out*
  - If you have asynchronous work coming in that you want to work on concurrently (generally in parallel), you can reach for the fan out pattern.
- *Fan in*
  - If you need to take work that has been fanned out and centralize it back into a single service, you can use the fan in pattern.

Talk the Walk

An important aspect of interviews is trying to convey how you reason about problems to your interviewer. You may be used to this from coding interviews, and the same applies for system design. You want to be explaining *why* you are making the decisions you are making, not just regurgitating memorized solutions.

In many system design interview frameworks there are two important sets of criteria you want to be sure to describe and agree on with your interviewer. *Functional* and *non-functional* requirements.

Functional requirements are the specifics about the feature or system you are designing. What does a simplified user flow look like? Who is the intended audience? Break this down into concrete bullet points and validate them with your interviewer.

Non-functional requirements are the behind the scenes things. What sort of scale are we designing for? What does read/write ratio look like? How do we handle adverse network conditions? These allow you to start building up the groundwork of your design.

Dazed and Confused

No technical interviewer worth their salt will expect you to have a 100% *perfect* solution. It's just not feasible given the time constraints. However, here are the things good interviewers *are* looking for:

- **Keep your cool**
  - Remember, you're talking to other people. At the end of the day just be kind and personable. Maybe crack a few jokes. Don't get into your own head about expectations.
- **Ask questions**
  - You often hear the term *clarifying questions* used. Those are the questions you ask upfront about the task. But what if you forget to ask one upfront? ***Ask anyway***. Showing you know when to stop and take a step back speaks leagues about your ability to engineer the *right* solution, not just *a* solution.
- **Know your limits**
  - You will undoubtedly reach a point where you don't know what to do. Rather than freeze or try to BS past it, say you're not sure. Explain what you might do in that situation (e.g. talk with more senior staff, read some documentation, etc).

Quick Maths

Engineering large systems requires a significant amount of forethought and planning. This often means doing some calculations based on historical data and making projections. You can't do that in an interview so here are some common things to think about:

- Remember to use units when writing numbers.
- Try to estimate primarily using orders of magnitude.
- Keep in mind common sizes of data types and common data representations.
- Don't forget to round. Your interviewer doesn't care if you can do long division.
- Common variables to solve for:
  - *Latency*
  - *Throughput*
  - *Storage*

Happy and Unhappy Paths

When you are designing pieces of these systems, always keep in mind how they can fail. You don't need to handle *every* possible scenario, but keep in mind common issues.

Most importantly, talk about *why* they are important. When you bring up a failure case don't just say what you will do to mitigate it, give a concrete reason why you are choosing that method over another one.

Time Marches On

As you start designing more and more distributed systems you will always hit a wall when it comes to syncing state between parts of your system. A common system for describing this is the *CAP theorem*. It states that any distributed data store can provide at most two of the following three guarantees:

- *Consistency*
  - Every read returns the most recent data or an error.
- *Availability*
  - Every request in the system that is handled by a healthy node **must** return a response, even if it isn't the most recent write.
- *Partition Tolerance*
  - The system can handle any number of dropped or high latency packets.

In modern networked systems you basically always require *partition tolerance*, so it becomes a tradeoff between *consistency* and *availability*. You may choose to allow for *eventual consistency* to achieve better availability, but may allow failures for data requests if you need more consistency.

Where to Go from Here

This is really just scratching the surface of designing larger systems. For next steps I highly recommend:

- *System Design Interview: An Insiders Guide* from Alex Xu
  - This two-part book series gives a great introduction and tons of relevant examples with clear explanations.
- *Designing Data Intensive Applications* from Martin Kleppmann
  - This is quite the read, but the insight into building large scale distributed systems is unparalleled.
- *Byte Byte Go*
  - This YouTube channel and email newsletter takes complex topics and gives you a high level understanding of them. This can allow you to get an idea of a topic that is tangential to something you are designing without having to *fully* understand it.
- *Exponent and Hello Interview*
  - These YouTube channels offer mock technical interviews that you can watch to better understand what these interviews look like, what interviewers are looking for, and what things to look out for when you get certain classes of questions.

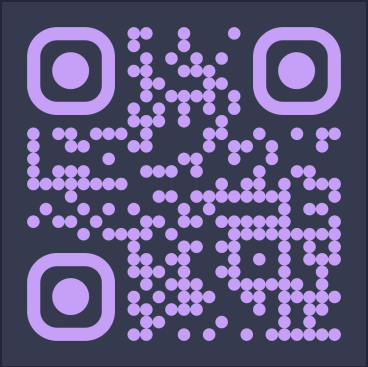
$: read questions

QUESTION TIME

$: which gvasquez

Please reach out with any questions! I truly love working on interesting computing problems, so I am always down to chat :D

My website! (*https://gvasquez.dev/*)



PHL Code Club (*https://phlcode.club/*)